CS489 Assignment 1 Writeup

Jeffrey Qin

October 13, 2026

This assignment was completed with assistance from generative AI.

- Any usage of AI in code generation or improvement has been described explicitly in a comment block at the top of each code file.
- Usage of AI in this write-up is limited to grammatical proofreading and syntax generation for tables.

1 Part 1

1.1 Subtask 1: Data Cleaning

Language	Raw train	Clean train	Raw dev	Clean dev
Finnish	17,029,153	10,058,581	1,678,953	1,007,027
Hungarian	17,379,397	10,317,997	1,790,559	1,036,953
German	18,347,716	9,836,130	1,749,689	996,285
Russian	30,104,329	17,259,456	3,137,332	1,703,770
Arabic	30,075,317	16,156,883	2,987,657	1,723,632
Mandarin Chinese	54,629,674	24,313,716	5,226,036	2,476,688
Vietnamese	21,539,469	12,455,974	2,208,053	1,284,544

The script uses a set of cleaning criteria universally applied to all languages:

- Remove HTML comments, as enclosed by <!–...–>.

- Remove reference tags, including self-closing tags in the form <ref name="..."/> and paired tags <ref>...</ref>.
- Remove template wrappers of the form {{...}}. Since there can be nested levels of template wrappers, the script runs multiple passes to remove occurrences from inner to outer levels.
- Remove table wrappers of the form {|...|}. All text enclosed by table wrappers are removed as well.
- Resolve Chinese variant switches -{...}-. Prioritize zh-hans, followed by zh-cn, followed by zh-sg.
- Remove link wrappers indicated by [...]. Wiki links of the form [[original name| shown name]] keeps the shown name. Links of the form [[name]] keeps name (for example, "[[nam]]es" becomes "names"). External links of the form [https://...com name] keeps the name. Category links [[Category:...]] and file links [[File:...]] are entirely removed.
- Remove bold (''') and italicize (') modifiers, as well as leading symbols like *, #, ;, and :.
- Remove heading wrappers that enclose text with at least two and at most six equal signs (e.g., ==, ======).
- Remove all leftover HTML pair tags, such as
,
, and ^{...}.
- Remove trailing or leading whitespace. Compress consecutive newlines so that there is at most a single blank line between paragraphs.

1.2 Subtask 2: Byte-level BPE Merge Loop

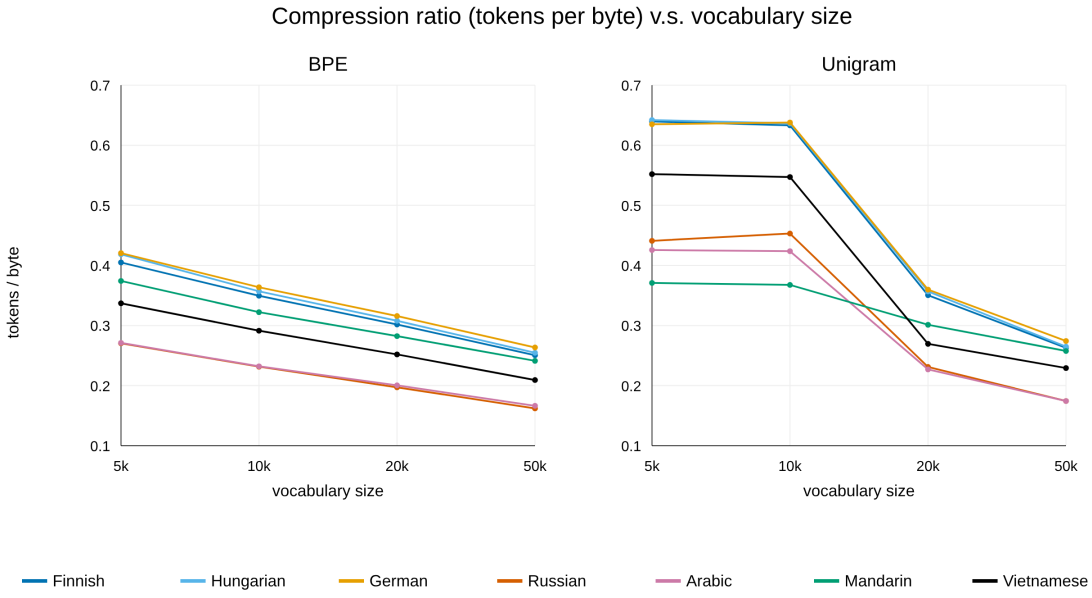
In my implementation of BPE, merges cannot cross line boundaries. As stated in the specifications, part (3) tokenizes each line independently, and so tokens that contain a newline (resulted from crossing line boundaries during vocabulary training) are never used at encoding time. Allowing such merges is wasteful by allocating vocabulary slots to tokens that encoding never produces.

1.3 Subtask 3: Encoder and Decoder

In byte-level BPE, all possible byte values (from 0 to 255) are included in the base vocabulary. Any input text is made of these byte values, even if it contains characters unseen in the training corpus. As a resulting, during encoding, any input byte sequence can be represented by tokens in the vocabulary. During decoding, the sequence of token IDs can be uniquely decoded to reconstruct the original input.

In character-level BPE, the base vocabulary only contains characters seen in the training corpus. When the input contains unseen characters, they are often encoded into a designated "unknown character" token. At decoding, however, each occurrence of the "unknown character" token does not remember which unseen character it maps to. Therefore, the original input cannot always be reconstructed.

1.4 Subtask 5: Analysis



For both BPE and unigram tokenizers, compression ratio decreased as vocabulary size increased, and converged to roughly the same tokens-per-byte count at V=50,000. Comparatively, BPE exhibited a more consistent decrease, which can be explained by their algorithmic differences

- BPE greedily and progressively generates new tokens based on the current vocabulary. Hence, larger vocabularies are simply smaller vocabularies with ad-

ditional merged tokens. These new merged tokens are built on existing tokens to further increase compression.

- Unigram probabilistically selects subwords to prune and keeps subwords that best explain the training data. As a result, changes to the target vocabulary size can substantially affect the subset of selected subwords and corpus segmentation, resulting in less smooth decrease in compression ratio.

Many observed features in the plot can be explained by morphological typology

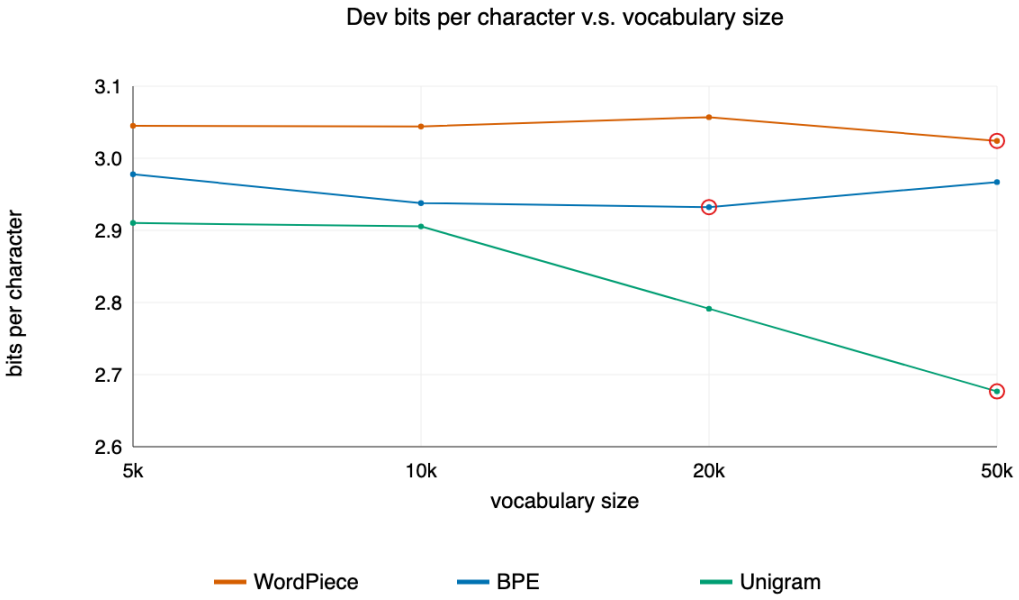
- Agglutinative languages, like Finnish and Hungarian, remain two of the least compressed languages. In agglutinative languages, multiple distinct morphemes are appended to the stem to express grammatical meaning, leading to a large number of possible surface forms. With limited vocabulary size, multiple tokens are needed to represent the vast combinations of morphemes.
- With the exception of Germany, fusional languages, like Russian and Arabic, achieved the most compression (lowest ratio). A single morpheme can carry multiple grammatical meanings, and so fewer tokens are needed to represent the morphological units associated with the stem. In Germany, we can combine multiple stems to form compounding long words, which leads to a large number of word forms that are not captured directly by merged tokens.
- Isolating languages, like Chinese and Vietnamese, have comparatively neutral compression ratios. Inflection is limited, and so the tokenizers do have as much of an issue with representing many possible surface word forms. However, grammartical features are represented by separate words, which may require additional tokens.

2 Part 2

2.1 Subtask 1: Baseline Run

The baseline dev bpc is 2.9729.

2.2 Subtask 3: the tokenization–compression tradeoff



Algorithm	V = 5,000	V = 10,000	V = 20,000	V = 50,000
BPE	2.9777	2.9379	2.9322	2.9670
Unigram	2.9103	2.9055	2.7914	2.6768
WordPiece	3.0451	3.0441	3.0570	3.0241

Based on observed trends in tokens-per-byte and bits-per-tokens over vocabulary sizes, better compression does not imply lower bpc. For both BPE and Unigram tokenizers, tokens-per-byte consistently decreases as vocabulary size increases. While the Unigram tokenizer still achieves lower bpc at larger vocabulary sizes, the trend is less noticeable. The same trend does not hold for the BPE tokenizer.

A possible explanation is the decomposition of bpc into bits-per-token and tokens-per-character. Given a larger vocabulary and better compression (lower tokens-per-character), text can be encoded with fewer number of tokens. As a result, individual

tokens are holding more information about the original text, increasing bits-per-token and consequently leveling out the decrease in tokens-per-character.

3 Part 3

3.1 Subtask 1: Maximum-likelihood n-gram model

A token-level n-gram LM was trained for each order $n \in \{2, 3, 4, 5\}$. No BOS (beginning-of-sequence) token is prepended.

- For order 2, the dev perplexity is $+\infty$, caused by the bigram (15029, 2357).
- For order 3, the dev perplexity is $+\infty$, caused by the trigram (2577, 13148, 1388).
- For order 4, the dev perplexity is $+\infty$, caused by the trigram (2577, 13148, 1388).
- For order 5, the dev perplexity is $+\infty$, caused by the trigram (2577, 13148, 1388).

The table below shows what each token ID involved in the unseen n-grams decodes back to.

Token Index	Token Piece
15029	" 29"
2357	" آ"
2577	"29"
13148	" أو"
1388	" مارس"

3.2 Subtask 2: Add-α smoothing

Table 1: Dev perplexity by order and α.

Order	0.00001	0.0001	0.001	0.01	0.1	1.0
2	2,515.050	1,536.140	1,014.339	986.798	2,034.387	7,253.538
3	11,579.510	5,916.988	5,513.606	8,671.137	16,220.055	29,130.857
4	18,940.762	16,247.380	17,943.790	22,660.456	29,902.632	38,663.382
5	25,721.279	24,620.982	26,372.396	30,086.905	35,416.948	41,445.016

Table 2: Dev bits-per-character by order and α.

Order	0.00001	0.0001	0.001	0.01	0.1	1.0
2	3.6956	3.4629	3.2670	3.2540	3.5955	4.1955
3	4.4162	4.0994	4.0660	4.2797	4.5753	4.8517
4	4.6485	4.5761	4.6230	4.7331	4.8640	4.9853
5	4.7929	4.7723	4.8047	4.8669	4.9439	5.0181

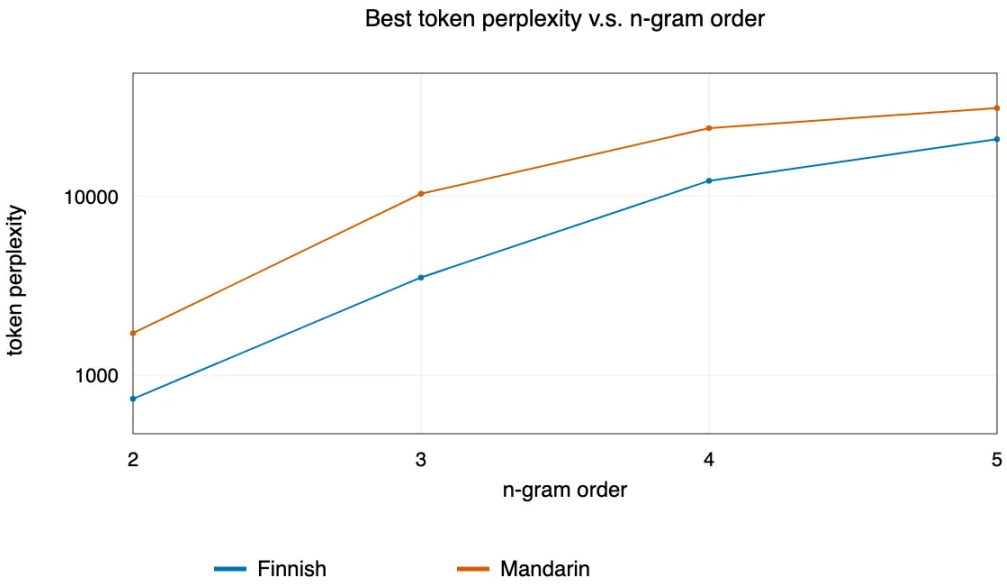
3.3 Subtask 3: Perplexity, bpc, and best-system selection

The optimal configuration for model training is $(n, \alpha) = (2, 0.01)$, as the pair achieves the minimum for both dev perplexity and bits-per-character. Mathematically, both implies that the model assigns the highest probability to the ground-truth next word on average among all (n, α) pairs.

Under the selected configuration, the dev perplexity and bpc are respectively 986.798 and 3.2540.

3.4 Subtask 4: Analysis

3.4.1 Part (a)



The plot illustrates best token perplexity (tuned on α) as a function of order. The model was trained on all languages combined and evaluated on the cleaned dev.txt for Finnish and Mandarin. In particular, higher order n-gram models exhibited higher perplexity, hence worsened performance. Two possible explanations are:

- Sparsity of long contexts in the training data. Since many long contexts are unseen, add-α smoothing causes many tokens to have an equally small probability of being predicted. Perplexity becomes high because the model has to predict blindly among these tokens.
- Language morphology. Finnish is agglutinative and relies on chaining additional tokens as grammatical modifiers. As order n increases, the context becomes an increasingly specific chain of modifiers, which is increasingly rare in the training data. Mandarin is isolating and comparatively, subsequent characters are more independent of previous characters. As a result, the next token is less predictable and perplexity starts higher at $n = 2$.

Add-α smoothing becomes more important as the order n increases. At lower orders, most contexts have been seen in training, so the model can perform most of its

predictions based on the counts it learned. At higher orders, many contexts are unseen, and any unsmoothed probability leads to infinite perplexity. For these unseen contexts, the model predicts entirely based on smoothing.

3.4.2 Part (b)

A larger BPE vocabulary would make add-α smoothing more important for a token-level n-gram model. Given a fixed order n, a larger vocabulary size would produce an exponentially larger number of distinct n-grams. Assuming that the training corpus is unchanged, each n-gram is counted fewer times and many n-grams that exist in the dev set can become unseen in the training corpus. Consequently, the model relies on smoothing more often in its predictions.